



ML to detect dementia

Scientists are using driving data and ML to detect dementia with a 66 percent success rate. <http://dgit.in/may21-05>



Youngest crypto billionaire

Amidst the crypto surge, Ethereum rose to \$3,200 making its creator the youngest crypto billionaire. <http://dgit.in/may21-06>

```
function getSquare(...numbers) {
  let result;
  for (let i = 0; i < numbers.length; i++) result
    = numbers[i] * numbers[i];
  console.log(result);
}

getSquare(1);
getSquare(1, 2);
getSquare(1, 2, 3, 4);
getSquare(1, 2, 3, 4, 6, 7);
```

1	script.js:4
4	script.js:4
16	script.js:4
49	script.js:4

its every call. However, the manner in which the argument is passed into the function is noteworthy here. We don't have any specific set of arguments mentioned. Instead, what we do have is a rest operator which receives an indefinite number of comma separated arguments and packs them into an array which we can use like any other. This becomes a very handy way of dealing with situations when we ourselves are not exactly sure of the number of parameters which we might send to the invoking function.

NULLISH COALESCING

Let's now look at the Nullish Coalescing operator which was introduced in ES2020. To establish the context for this feature of JavaScript, let's consider that we have an event where the user has to enter the number of players taking part in a competition. If the player does not enter any value, we default the value to a specific number just for the sake of clarity. We can do this by a simple if-else structure but what happens to be the point of using that if we're not leveraging the usefulness of the modern ES6 operators.

```
const noOfEntries = prompt('Enter number of
players');
const defaultPlayers = 11;
const finalEntries = noOfEntries ||
defaultPlayers;
console.log(finalEntries);
```

The user may or may not enter a value, which is then stored in noOfEntries. We set finalEntries as the logical

OR of noOfEntries and defaultPlayers. What we are trying to achieve here is the functionality to set finalEntries to the default

value in case the user doesn't enter any value at all, since that is the way logical OR works.

It's going to take a look at noOfEntries (which in our case is null), hence will skip that and will look into whether defaultPlayers has null or otherwise. In this case, there's a definite value assigned to defaultPlayers and it'll set this value to finalEntries.

There is but one major problem in this manner of the code. If the user enters 0, and well, there could be a situation where there can be no players at all. But, the logical OR will treat this as a falsy value as JavaScript considers 0 or an empty string to be nullish values. The execution will flow forward and it will pick the defaultPlayers again and set it to finalEntries, thereby, breaking the logic. We had to somehow make JavaScript understand that 0 in this case is an acceptable value and definitely, an intended value too.

ES2020 gave us the nullish coalescing operator which saves the day under such circumstances. The operator appears like ?? and it works almost in the same way the logical OR does, except that it doesn't treat 0 or a blank space to be nullish. Only null and undefined values are treated as

nullish when it comes to this operator, which goes to say that if we have

**operand1 ??
operand2;**

The result will not short-circuit unless either one of the operands is either null or undefined. If we simply replace the line of code for finalEntries in the above example as-

```
const finalEntries = noOfEntries ??
defaultPlayers
```

```
const noOfEntries = prompt('Enter number of
players');
const defaultPlayers = 11;
const finalEntries = noOfEntries ??
defaultPlayers;
console.log('you have entered ' + finalEntries);
```

it would mean that if the user enters 0 as the noOfEntries, inherently 0 will be considered as a truth value and will instantly short-circuit the operation and finalEntries would be set to 0. It becomes very useful while taking care of corner-cases and other tricky use-cases that might lead up to some error, locating which might get very tedious due to the nature of the errors of such kind.

THERE'S MORE

There obviously are way more features in ES6, and as it so happens, every year a bunch of new additions are introduced into the EcmaScript standard that gives JavaScript the immense popularity for it to thrive on with a crazy learning curve. Some of the features get deprecated and some gets overused. But as they say, some features stand the test of time due to their usefulness, simplicity and overwhelming community support and some features are exponentially added on. One such example would be the rest and spread operators which were introduced in ES6 and were later extended to support rest and spread properties introduced in the year 2018. The above-mentioned features are some of the widely renowned and go-to tools for the majority of the use-cases that suit their needs and would not need further workarounds until some much easier pops up in the future. That being said, it is true that sometimes, the changes introduced get too quirky or complex for certain developers but then all of it comes down to personal preferences and comprehensible coding styles. ➡